

Unit 5 Coding Reference Manual

Unit 5 Coding Reference Manual

Docker, Docker Compose and containerising Django, the commands and files you will use in every lab and project

This manual collects the Docker commands and files used in Unit 5 into one document. It uses the Unit 2 Django project as the running example throughout: one Dockerfile and one compose file, built up section by section. Treat it as the one place to look things up while working. The ideas behind each command are in the Unit 5 Notes.

NOT A REPLACEMENT FOR THE CODE FILES SHARED IN THE DEMO FOLDER OF EACH LAB

Contents

1. Setup
2. Images
3. Containers
4. The Dockerfile
5. Preparing Django for Docker
6. Volumes and Networks
7. Docker Compose
8. Registries and Deployment
9. Debugging and Clean Up

Windows note: commands work in PowerShell or Git Bash. Where a command is split across lines with `\`, use a backtick in PowerShell, or write it on one line.

1. Setup

Docker Desktop must be running (the whale icon in the system tray) before any command works.

Command	Purpose
<code>docker version</code>	Shows client and daemon versions; an error means Docker Desktop is not started
<code>docker compose version</code>	Confirms Compose is installed

Command	Purpose
<code>docker run hello-world</code>	Pulls and runs a test image, proving the installation works

```
docker run hello-world
```

```
docker ps -a      # the finished container, status Exited (0)
```

```
docker images    # the image stays in the local cache
```

2. Images

Command	Purpose
<code>docker build -t name:tag .</code>	Builds an image from the Dockerfile in the current folder
<code>docker build -f Dockerfile.prod -t name:tag .</code>	Builds from a Dockerfile with a different name
<code>docker images</code>	Lists local images and their sizes
<code>docker history name:tag</code>	Shows each layer and its size
<code>docker pull postgres:16-alpine</code>	Downloads an image from a registry
<code>docker tag name:tag user/name:tag</code>	Gives an image another name, before pushing
<code>docker push user/name:tag</code>	Uploads an image to a registry
<code>docker rmi name:tag</code>	Removes an image

```
docker build -t imap-django:1.0 .
```

```
docker history imap-django:1.0    # which layers are large?
```

Base image	Size	Use it for
<code>python:3.12</code>	1.62 GB	Only build stages that need compilers
<code>python:3.12-slim</code>	190 MB	The default for Python apps
<code>alpine:3.20</code>	12 MB	Tiny utilities; some Python packages will not install

Always pin at least `major.minor` in a tag. Never rely on `latest`.

3. Containers

3.1 docker run options

Option	Purpose
<code>-d</code>	Runs in the background
<code>-p 8000:8000</code>	Publishes a port, HOST:CONTAINER
<code>--name web</code>	A readable name instead of a random ID
<code>-e KEY=value</code>	Sets an environment variable
<code>--env-file .env</code>	Loads environment variables from a file
<code>-v name:/path</code>	Mounts a named volume
<code>--network net</code>	Joins a user defined network
<code>-it</code>	Interactive terminal, for shells
<code>--rm</code>	Removes the container when it exits
<code>--restart unless-stopped</code>	Restarts after a crash or reboot

Options go **before** the image name. Anything **after** it replaces the image's CMD.

```
docker run -d -p 8080:80 --name web nginx:alpine      # open http://localhost:8080
docker run -it --rm python:3.12-slim bash           # a throwaway shell
```

3.2 Everyday commands

Command	Purpose
<code>docker ps</code> / <code>docker ps -a</code>	Running containers / all, including exited
<code>docker logs -f web</code>	Follows a container's output
<code>docker exec -it web sh</code>	Opens a shell inside a running container
<code>docker inspect web</code>	Ports, env vars, mounts, network, health
<code>docker stats</code>	Live CPU and memory use
<code>docker stop web</code> / <code>docker start web</code>	Stop gracefully / start again
<code>docker rm web</code> / <code>docker rm -f web</code>	Remove a stopped / running container

Exit codes in `docker ps -a`: `0` finished normally, `1` the app crashed, `127` command not found, `137` killed, usually out of memory.

4. The Dockerfile

4.1 Instructions

Instruction	Purpose
<code>FROM</code>	Base image; every Dockerfile starts here
<code>WORKDIR</code>	Sets and creates the working directory
<code>COPY</code>	Copies files from the project into the image
<code>RUN</code>	Runs a command at build time, saved as a layer
<code>ENV</code>	Environment variable, at build and run time
<code>ARG</code>	Variable available only during the build
<code>EXPOSE</code>	Documents the port; publishes nothing
<code>USER</code>	Runs everything after it as this user
<code>CMD</code>	Default command; replaced by arguments to <code>docker run</code>
<code>ENTRYPOINT</code>	Fixed program; CMD becomes its default arguments
<code>HEALTHCHECK</code>	Command Docker runs to check the app is healthy

4.2 The Django Dockerfile

```
# Dockerfile
FROM python:3.12-slim # 1

ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 # 2

WORKDIR /app # 3

COPY requirements.txt . # 4
RUN pip install --no-cache-dir -r requirements.txt

COPY . . # 5
RUN python manage.py collectstatic --noinput # 6

RUN useradd --create-home --uid 10001 appuser \
  && chown -R appuser /app # 7
USER appuser

EXPOSE 8000 # 8
CMD ["gunicorn", "mysite.wsgi:application", "--bind", "0.0.0.0:8000", "--workers",
"2"]
```

Line	Why
1	A pinned, slim base image

Line	Why
2	No <code>.pyc</code> files; logs appear immediately in <code>docker logs</code>
3	Every later path is relative to <code>/app</code>
4	Requirements first, so the slow install layer stays cached
5	Code last, since it changes most often
6	Gathers static files so the admin site is styled with <code>DEBUG</code> off
7	Runs the app as a non root user
8	gunicorn serves <code>mysite/wsgi.py</code> on <code>0.0.0.0</code> , reachable from outside the container

4.3 .dockerignore

```
.git
venv/
__pycache__/
*.pyc
.env
db.sqlite3
staticfiles/
Dockerfile*
compose*.yaml
```

Keeps the venv, Git history and passwords out of the image. Do **not** add `*.pkl`: the trained model must travel inside the image.

4.4 Useful variations

Want	Change
A health status in <code>docker ps</code>	Add <code>HEALTHCHECK</code> <code>CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:8000/admin/login/')"</code>
A fixed program with changeable arguments	<code>ENTRYPOINT ["ping", "-c", "3"]</code> plus <code>CMD ["localhost"]</code> ; <code>docker run img google.com</code> replaces only <code>CMD</code>
Build tools kept out of the final image	Multi stage: <code>FROM python:3.12 AS builder</code> installs, then <code>COPY --from=builder</code> into a slim final stage

Never put secrets in ENV or ARG. `docker inspect` shows them to anyone who has the image.

5. Preparing Django for Docker

5.1 requirements.txt

```
Django==5.0.6
django-rest-framework==3.15.1
psycopg2-binary==2.9.9
gunicorn==23.0.0
whitenoise==6.7.0
scikit-learn==1.5.0
joblib==1.4.2
```

These versions are examples. Generate the real list from your working venv with `pip freeze > requirements.txt`, so scikit-learn matches the version that trained `iris_model.pkl`.

5.2 settings.py changes

```
# mysite/settings.py
import os

SECRET_KEY = os.environ.get("DJANGO_SECRET_KEY", "dev-only-not-secret")
DEBUG = os.environ.get("DJANGO_DEBUG", "0") == "1"
ALLOWED_HOSTS = os.environ.get("DJANGO_ALLOWED_HOSTS",
                                "localhost,127.0.0.1").split(",")

MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "whitenoise.middleware.WhiteNoiseMiddleware",      # add this line here
    # ... the rest unchanged ...
]

DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": os.environ.get("POSTGRES_DB", "imapdb"),
        "USER": os.environ.get("POSTGRES_USER", "postgres"),
        "PASSWORD": os.environ.get("POSTGRES_PASSWORD", ""),
        "HOST": os.environ.get("POSTGRES_HOST", "localhost"),
        "PORT": "5432",
    }
}

STATIC_ROOT = BASE_DIR / "staticfiles"
```

The defaults keep `python manage.py runserver` on your laptop working exactly as in Unit 2. Inside Compose, `.env` and `POSTGRES_HOST=db` override them.

6. Volumes and Networks

Command	Purpose
<code>docker volume create pgdata</code>	Creates a named volume
<code>docker volume ls</code> / <code>inspect pgdata</code>	Lists volumes / shows where one lives
<code>docker volume rm pgdata</code>	Deletes a volume and its data
<code>docker network create imap-net</code>	Creates a network with name based DNS
<code>docker network inspect imap-net</code>	Shows which containers are attached

Running the Django stack by hand shows what Compose automates in section 7:

```
docker network create imap-net
docker volume create pgdata

docker run -d --name db --network imap-net \
  -e POSTGRES_PASSWORD=secret -e POSTGRES_DB=imapdb \
  -v pgdata:/var/lib/postgresql/data postgres:16-alpine

docker run -d --name web --network imap-net -p 8000:8000 \
  -e POSTGRES_HOST=db -e POSTGRES_PASSWORD=secret \
  imap-django:1.0
```

From	Reach PostgreSQL at
Your laptop, port published	<code>localhost:5432</code>
Another container on the same network	<code>db:5432</code>
A container, reaching your laptop	<code>host.docker.internal:5432</code>

7. Docker Compose

7.1 .env

```
# .env (never committed; commit a .env.example with placeholders)
DJANGO_SECRET_KEY=replace-with-a-long-random-string
DJANGO_DEBUG=0
DJANGO_ALLOWED_HOSTS=localhost,127.0.0.1
POSTGRES_USER=postgres
POSTGRES_PASSWORD=change-me-please
POSTGRES_DB=imapdb
```

7.2 compose.yaml

```

services:
  web:
    build: .
    image: imap-django:1.0
    command: sh -c "python manage.py migrate && gunicorn mysite.wsgi:application --
bind 0.0.0.0:8000 --workers 2"
    ports:
      - "8000:8000"
    env_file: .env
    environment:
      POSTGRES_HOST: db
    depends_on:
      db:
        condition: service_healthy
    restart: unless-stopped

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 5s
      retries: 10

volumes:
  pgdata:

```

Key	Purpose
<code>build</code> / <code>image</code>	Build from the Dockerfile, and name the result
<code>command</code>	Runs migrations, then gunicorn, every time web starts
<code>ports</code>	<code>"HOST:CONTAINER"</code> , always quoted
<code>env_file</code> / <code>environment</code>	Variables from <code>.env</code> / set inline
<code>POSTGRES_HOST: db</code>	The service name, never localhost
<code>depends_on: service_healthy</code>	Web waits until PostgreSQL is actually ready
<code>volumes</code>	Database files live in the named volume <code>pgdata</code>

Key	Purpose
<code>\$\${...}</code>	Escapes <code>\$</code> so Compose leaves it for the container's shell

The database publishes no port, so only the web container can reach it, and it will not clash with an `imap-postgres` container from Unit 2.

7.3 Commands

Command	Purpose
<code>docker compose up -d --build</code>	Builds and starts everything in the background
<code>docker compose ps</code>	Status of each service
<code>docker compose logs -f web</code>	Follows the web service's logs
<code>docker compose exec web python manage.py createsuperuser</code>	Creates an admin account inside the container
<code>docker compose exec web python manage.py drf_create_token <user></code>	Creates an API token
<code>docker compose exec db psql -U postgres -d imapdb</code>	SQL prompt inside the database
<code>docker compose config</code>	Prints the final YAML with <code>.env</code> filled in
<code>docker compose down</code>	Stops everything, keeps the database
<code>docker compose down -v</code>	Also deletes the volume and all data

```

cp .env.example .env                                # PowerShell: copy .env.example .env
docker compose up -d --build
docker compose exec web python manage.py createsuperuser
# open http://localhost:8000/admin/

curl -X POST http://localhost:8000/api/predict/ \
  -H "Authorization: Token <your-token>" \
  -H "Content-Type: application/json" \
  -d '{"sepal_length": 5.1, "sepal_width": 3.5, "petal_length": 1.4,
"petal_width": 0.2}'
# {"prediction": 0}

```

After changing code, `docker compose up -d --build` rebuilds and restarts only what changed. After changing a model, run `makemigrations` on your laptop as in Unit 2, then rebuild; the container applies the migration on start.

8. Registries and Deployment

8.1 Push and pull

```
# on your laptop
docker login
docker tag imap-django:1.0 <your-hub-user>/imap-django:1.0
docker push <your-hub-user>/imap-django:1.0

# on the server: copy compose.yaml and .env, change build: . to
#   image: <your-hub-user>/imap-django:1.0
docker compose up -d
```

Tag	Use it for
1.0, 1.4.2	Releases; never reuse a version tag
sha-3f2a9c1	Tracing an image to its exact commit
latest	Convenience only, never production

Built on an Apple M series Mac and deploying to an Intel or AMD server? Build for both:

```
docker buildx build --platform linux/amd64,linux/arm64 \
  -t <your-hub-user>/imap-django:1.0 --push .
```

8.2 CI/CD with GitHub Actions

```
# .github/workflows/docker-publish.yml
name: docker-publish
on:
  push:
    branches: [main]

jobs:
  docker:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-buildx-action@v3
      - uses: docker/login-action@v3
      with:
        registry: ghcr.io
```

```

    username: ${GITHUB_ACTOR}
    password: ${GITHUB_TOKEN}
- uses: docker/build-push-action@v6
  with:
    context: .
    push: true
    tags: ghcr.io/${GITHUB_REPOSITORY}:latest

```

Every push to `main` builds the image and pushes it to `ghcr.io/<you>/<repo>`. GitHub supplies the token automatically. Repository names must be lowercase.

9. Debugging and Clean Up

Step	Command	Look for
1	<code>docker compose ps</code> or <code>docker ps -a</code>	Running, exited, restarting; the exit code
2	<code>docker compose logs web</code>	The actual error
3	<code>docker inspect <c></code>	Ports, env vars, mounts, network
4	<code>docker compose exec web sh</code>	Look around from inside
5	<code>docker compose config</code>	Variables filled in correctly?

Error	Fix
<code>port is already allocated</code>	Change the host port (<code>"8001:8000"</code>) or stop the other container
<code>connection refused</code> / <code>could not translate host name "db"</code>	Use <code>db</code> , not <code>localhost</code> ; check <code>POSTGRES_HOST</code> and that both services are in the same compose file
<code>DisallowedHost</code>	Add the host to <code>DJANGO_ALLOWED_HOSTS</code>
Admin page has no styling	WhiteNoise missing from <code>MIDDLEWARE</code> , or <code>collectstatic</code> did not run
<code>Database is uninitialized and superuser password is not specified</code>	Set <code>POSTGRES_PASSWORD</code> in <code>.env</code>
<code>exec format error</code>	Rebuild with <code>docker buildx build --platform ...</code>
<code>Exited (137)</code>	Out of memory
App unreachable but container running	App bound to <code>127.0.0.1</code> ; bind to <code>0.0.0.0</code>

Command	Removes
<code>docker system df</code>	Nothing; shows disk use
<code>docker system prune</code>	Stopped containers, unused networks, dangling images, build cache

Command	Removes
<code>docker system prune -a --volumes</code>	Everything unused , including images and volumes

Quick tips: pin a slim base, never `latest`. Requirements before code. Bind to `0.0.0.0`. Secrets in `.env`, never in the image. Service names, not localhost. Think twice before `down -v`.